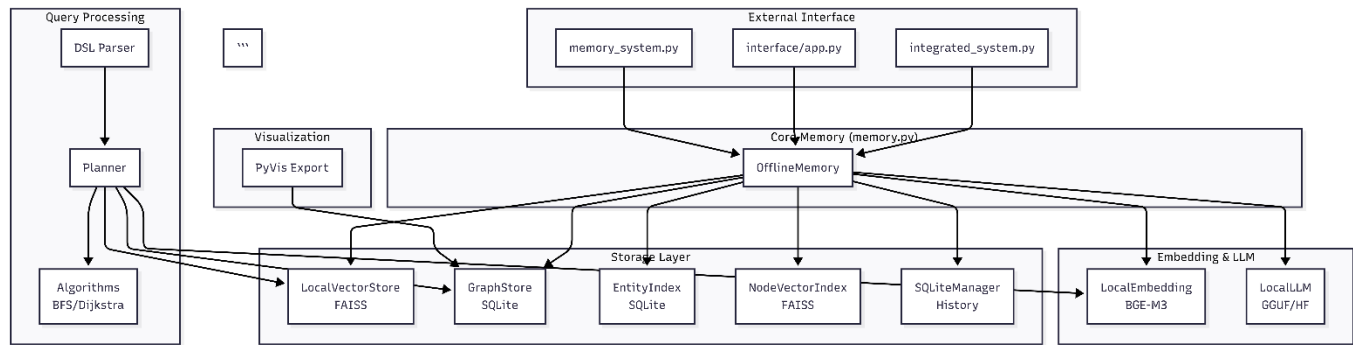


Amem Memory Architecture Diagram

Overview

Amem is a hybrid memory system combining **vector-based semantic search** (FAISS + BGE-M3) with **graph-based relational reasoning** (SQLite property graph). It supports offline operation with local LLMs.



File-by-File Architecture

1. memory.py - Core Orchestrator (1604 lines)

Class: OfflineMemory - Main entry point for all memory operations.

Method	Purpose	Dependencies
<code>init()</code>	Initialize all storage components	<code>LocalVectorStore</code> , <code>GraphStore</code> , <code>EntityIndex</code> , <code>LocalEmbedding</code> , <code>LocalLLM</code>
<code>add()</code>	Store memory to FAISS + Graph	<code>_index_entities_and_graph()</code> , <code>vectors.add()</code>
<code>search()</code>	Hybrid retrieval (vector + graph + rerank)	<code>vectors.search()</code> , <code>graph.k_hop()</code> , <code>traverse()</code> , <code>embedder.graph_enhanced_rerank_m3()</code>
<code>traverse()</code>	A* graph pathfinding	<code>graph.neighbors()</code> , <i>possessive chain parsing</i>
<code>update()</code>	Update existing memory	<code>vectors.update()</code>

ARASHA Memory System

<i>delete()</i>	<i>Remove memory and relations</i>	<i>vectors.delete(), graph.delete_by_memory_id()</i>
<i>delete_by_query()</i>	<i>Delete memories matching query</i>	<i>search(), delete()</i>
<i>history()</i>	<i>Get memory change history</i>	<i>db.get_history()</i>
<i>rebuild_vector_index()</i>	<i>Rebuild FAISS from docstore</i>	<i>vectors.rebuild_from_docstore()</i>

Private Methods:

Method	Purpose
<i>_index_entities_and_graph()</i>	<i>Extract entities via regex/LLM, create graph edges</i>
<i>_merge_similar_entities_and_relationships()</i>	<i>Merge similar entities by embedding cosine similarity</i>
<i>_update_faiss_entity_references()</i>	<i>Update FAISS payloads when entities merge</i>
<i>_make_payload()</i>	<i>Create vector payload with metadata</i>
<i>_cache_neighbors()</i>	<i>LRU cache for graph neighbor lookups</i>
<i>build_comprehensive_graph_context()</i>	<i>Build rich context for reranking</i>

2. graph_store.py - SQLite Property Graph (762 lines)

Class: GraphStore - SQLite-backed graph with nodes and edges.

Schema:

nodes(id, name, label, props, cluster_id, created_at, updated_at)

edges(source, relationship, destination, source_id, destination_id, memory_id, weight, created_at, updated_at)

edge_props(source, relationship, destination, key, value)

Method	Purpose
<i>add(rel)</i>	<i>Insert edge</i>
<i>upsert(rel)</i>	<i>Insert or update edge</i>
<i>delete(src, rel, dst)</i>	<i>Remove specific edge</i>
<i>delete_by_memory_id(mid)</i>	<i>Remove all edges for a memory</i>
<i>neighbors(entity, direction, limit)</i>	<i>Get 1-hop neighbors</i>
<i>k_hop(entity, k, limit_per_hop)</i>	<i>Multi-hop BFS expansion</i>

ARASHA Memory System

<i>query_edges(relationship, source, destination)</i>	<i>Filter edges</i>
<i>query_nodes(limit)</i>	<i>Get all nodes</i>
<i>find_similar_entities(name, threshold)</i>	<i>String similarity search</i>
<i>merge_entities(source, target)</i>	<i>Merge two entities</i>
<i>merge_relationships(source, target)</i>	<i>Merge relationship types</i>
<i>get_edge_weight(a, b)</i>	<i>Get edge weight</i>
<i>set_edge_prop(src, rel, dst, key, val)</i>	<i>Set edge property</i>
<i>get_edge_prop(src, rel, dst, key)</i>	<i>Get edge property</i>
<i>upsert_node()</i>	<i>Create/update node</i>
<i>ensure_node()</i>	<i>Get or create node</i>
<i>recompute_clusters()</i>	<i>Union-find clustering</i>

Data Class: Relation

@dataclass

class Relation:

source: str
relationship: str
destination: str
created_at: str
updated_at: str
weight: float = 1.0
memory_id: str | None = None

3. vector_store.py - FAISS Vector Store (220 lines)

Class: LocalVectorStore - FAISS-backed dense vector index.

Method	Purpose
<i>add(vectors, payloads, ids)</i>	<i>Add vectors with metadata</i>
<i>update(id, vector, payload)</i>	<i>Update existing vector</i>
<i>delete(id)</i>	<i>Remove vector</i>
<i>get(id)</i>	<i>Get vector record</i>
<i>search(query_vec, limit, filters)</i>	<i>Similarity search</i>
<i>list(limit)</i>	<i>List all IDs</i>
<i>items()</i>	<i>Iterator over (id, payload)</i>
<i>rebuild_from_docstore()</i>	<i>Rebuild index from stored payloads</i>
<i>reset_index()</i>	<i>Clear and recreate index</i>

4. local_embedding.py - BGE-M3 Embeddings (426 lines)

Class: LocalEmbedding - Multi-modal embedding using BGE-M3.

ARAISHA Memory System

Method	Purpose	Output
<code>embed(text)</code>	Single text → dense vector	List[float] (1024-dim)
<code>embed_many(texts)</code>	Batch embedding	List[List[float]]
<code>embed_with_lexical(texts)</code>	Dense + sparse (BM25)	{"dense": [...], "lexical": [...]}
<code>lexical_score(q_lex, d_lex)</code>	BM25-like matching score	float
<code>pair_scores_m3(pairs)</code>	Dense/Sparse/ColBERT scores	Dict[str, List[float]]
<code>advanced_rerank_m3(query, docs)</code>	Multi-fusion reranking	List[Dict] with breakdown
<code>graph_enhanced_rerank_m3(query, memories, contexts)</code>	Graph-aware reranking	List[Dict] with graph boosts

Scoring Strategies:

- Dense (semantic similarity)
- Sparse (BM25-like lexical)
- ColBERT (fine-grained token matching)
- Fusion: Linear, Geometric, Harmonic, Max

5. entity_index.py - Entity-Memory Mapping (91 lines)

Class: EntityIndex - SQLite index linking entities to memories.

Schema:

```
entities(entity TEXT PRIMARY KEY)
aliases(alias TEXT PRIMARY KEY, canonical TEXT)
memory_entities(memory_id TEXT, entity TEXT)
```

Method	Purpose
<code>add_memory_entities(mid, text, extra)</code>	Extract & store entities
<code>entities_for_memory(mid)</code>	Get entities linked to memory
<code>memories_for_entity(entity)</code>	Get memories mentioning entity
<code>add_alias(alias, canonical)</code>	Create alias mapping
<code>resolve(name)</code>	Resolve alias to canonical

Entity Pattern: `r"\b([A-Z][a-zA-Z0-9_\-]+)\b"` (proper-case words)

6. local_llm.py - Local LLM Backend (380 lines)

Class: LocalLLM - Unified interface for GGUF and HuggingFace models.

Method	Purpose
<code>generate(prompt)</code>	One-shot generation

Method	Purpose
<code>generate_messages(messages)</code>	Chat completion
<code>generate_messages_stream(messages)</code>	Streaming chat

Backends:

- **GGUF** (llama-cpp-python) - Quantized models
- **Transformers** (HuggingFace) - Standard models

Message Formatting: Llama 3.1 Instruct format with special tokens.

7. local_config.py - Configuration (185 lines)

Configuration Classes:

Class	Purpose	Key Fields
<code>LocalLLMConfig</code>	LLM settings	<code>model_path</code> , <code>backend</code> , <code>max_new_tokens</code>
<code>LocalEmbedderConfig</code>	Embedder settings	<code>model_path</code> (bge-m3), <code>embedding_dims</code> (1024)
<code>LocalVectorStoreConfig</code>	FAISS settings	<code>path</code> , <code>index_type</code> (HNSW), <code>distance_strategy</code>
<code>LocalGraphConfig</code>	Graph DB path	<code>db_path</code> (graph.db)
<code>EntityIndexConfig</code>	Entity DB path	<code>db_path</code> (entity_index.db)
<code>RetrievalConfig</code>	Search params	<code>initial_top_k</code> , <code>final_top_k</code> , <code>scoring weights</code>
<code>RerankerConfig</code>	Reranker params	<code>top_k</code> , <code>max_passage_length</code> , <code>enabled</code>
<code>ScoringWeights</code>	Hybrid scoring	<code>w_similarity</code> , <code>w_graph</code> , <code>w_recency</code> , <code>w_evidence</code>
<code>OfflineMemoryConfig</code>	Master config	Aggregates all above

8. storage.py - History Management (219 lines)

Class: SQLiteManager - Memory change history.

<i>Method</i>	<i>Purpose</i>
<i>add_history(mid, old, new, event)</i>	<i>Log memory change</i>
<i>get_history(mid)</i>	<i>Get change log</i>
<i>reset()</i>	<i>Clear history</i>

Schema:history(id, memory_id, old_memory, new_memory, event, created_at, updated_at, is_deleted, actor_id, role)

9. algorithms.py - Graph Algorithms (91 lines)

<i>Function</i>	<i>Purpose</i>
<i>shortest_path(graph, start, goal, max_depth)</i>	<i>BFS shortest path</i>
<i>dijkstra_shortest_path(graph, start, goal)</i>	<i>Weighted Dijkstra</i>
<i>path_to_relations(graph, path)</i>	<i>Convert node path to edges</i>

10. planner.py - DSL Query Executor (103 lines)

Class: Planner - Executes parsed DSL queries.

<i>Method</i>	<i>Purpose</i>
<i>execute(query)</i>	<i>Run query against graph+vectors</i>

Query Flow:

1. Vector prefilter via SIMILAR() clause
 2. Graph expansion by label/name
 3. Property filter (WHERE)
 4. Shortest path if multi-hop
 5. Return matching rows
-

11. dsl.py - Query DSL Parser (144 lines)

Data Classes:

- NodePat: Node pattern
-

ARAISHA Memory System

(var:Label {props})

- RelPat: Relationship pattern -[:TYPE*min..max]->
- MatchPattern: Full match clause
- WherePredicate: WHERE conditions
- Query: Complete parsed query

Function	Purpose
-----------------	----------------

<i>parse(query)</i>	<i>Parse Cypher-like DSL to Query</i>
---------------------	---------------------------------------

DSL Syntax:

```
MATCH (a:Person)-[:FRIEND*1..3]->(b:Person)
WHERE a.name='Alice' AND SIMILAR(b, 'engineer', 10, 0.5)
RETURN a, b LIMIT 25
```

12. node_vectors.py - Node Vector Index (48 lines)

Class:

NodeVectorIndex - Separate FAISS index for graph nodes.

Method	Purpose
<i>upsert(node_id, vector, payload)</i>	<i>Store node embedding</i>
<i>search(vector, limit)</i>	<i>Find similar nodes</i>
<i>delete(node_id)</i>	<i>Remove node vector</i>
<i>get(node_id)</i>	<i>Get node payload</i>

Used for fast graph node similarity search.

13. visualize.py - Graph Visualization (274 lines)

Function	Purpose
<i>export_pyvis(graph, nodes, edges, out_dir)</i>	<i>Generate interactive HTML graph</i>

14. memory_system.py - Alternative Entry Point (1264 lines)

Standalone CLI with similar functionality to integrated_system.py.

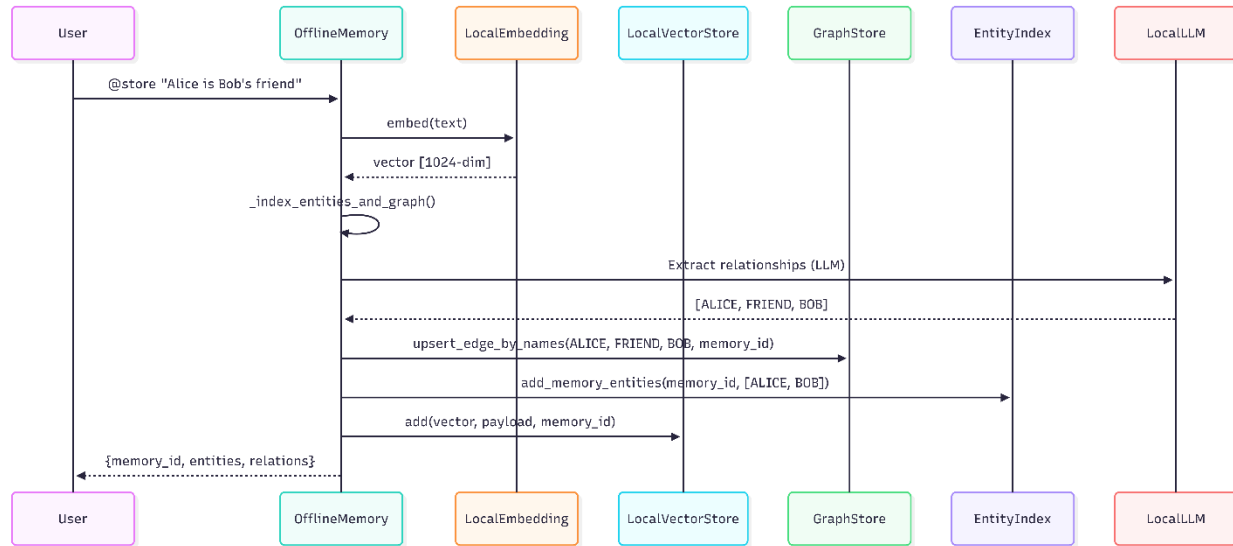
Function	Purpose
<i>optimize_memory_search()</i>	<i>Optimized hybrid search</i>
<i>_get_memory_context()</i>	<i>Build augmentation context</i>
<i>_semantic_query_expansion()</i>	<i>Expand query with synonyms</i>
<i>_adaptive_search_strategy()</i>	<i>Query-dependent params</i>
<i>_multi_hop_reasoning()</i>	<i>Chain-of-thought retrieval</i>
<i>parse_action()</i>	<i>Parse @commands</i>
<i>main()</i>	<i>CLI entry point</i>

15. utils.py - Utilities (25 lines)

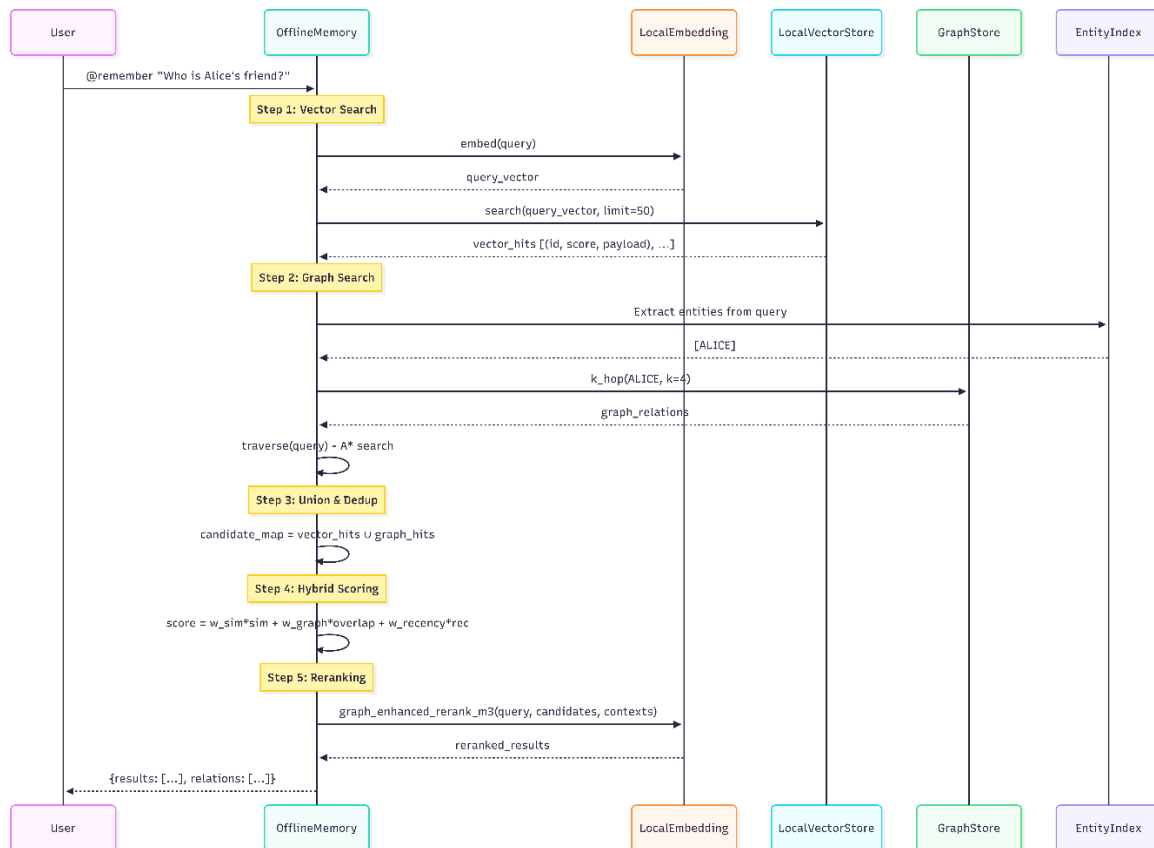
Function	Purpose
<i>utc_now_iso()</i>	<i>Current UTC timestamp</i>
<i>md5_hash(text)</i>	<i>Text hash for dedup</i>
<i>merge_texts(a, b)</i>	<i>Combine unique tokens</i>

Data Flow Diagrams

Storage Flow (@store)

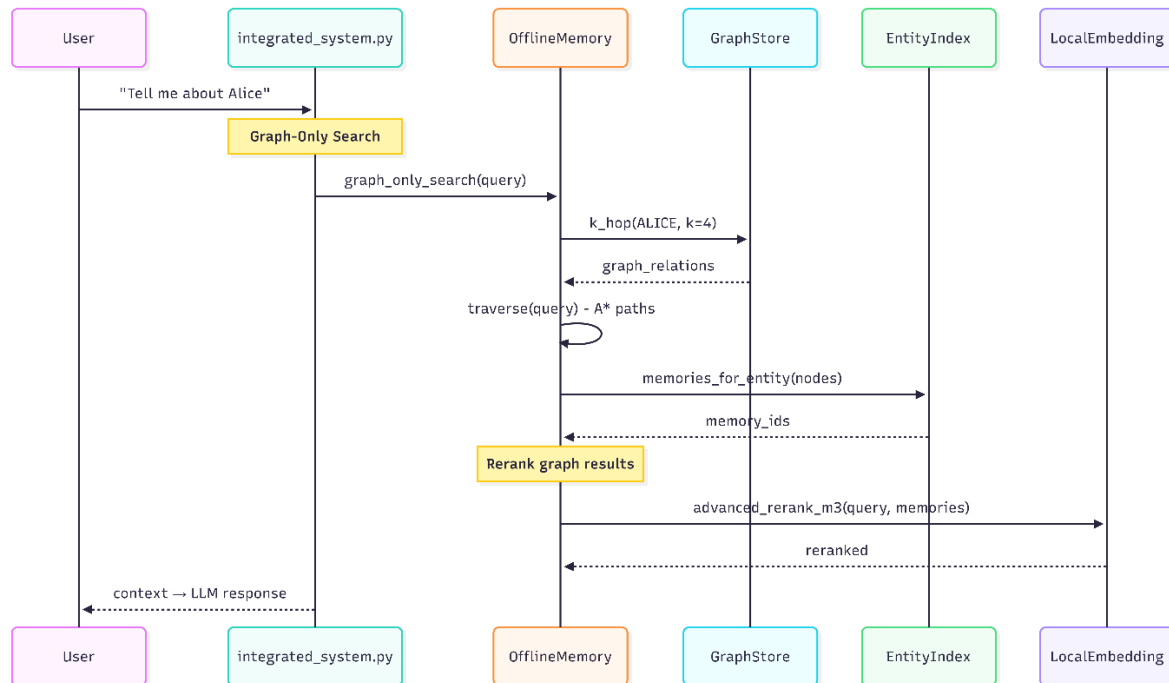


Retrieval Flow (@remember)

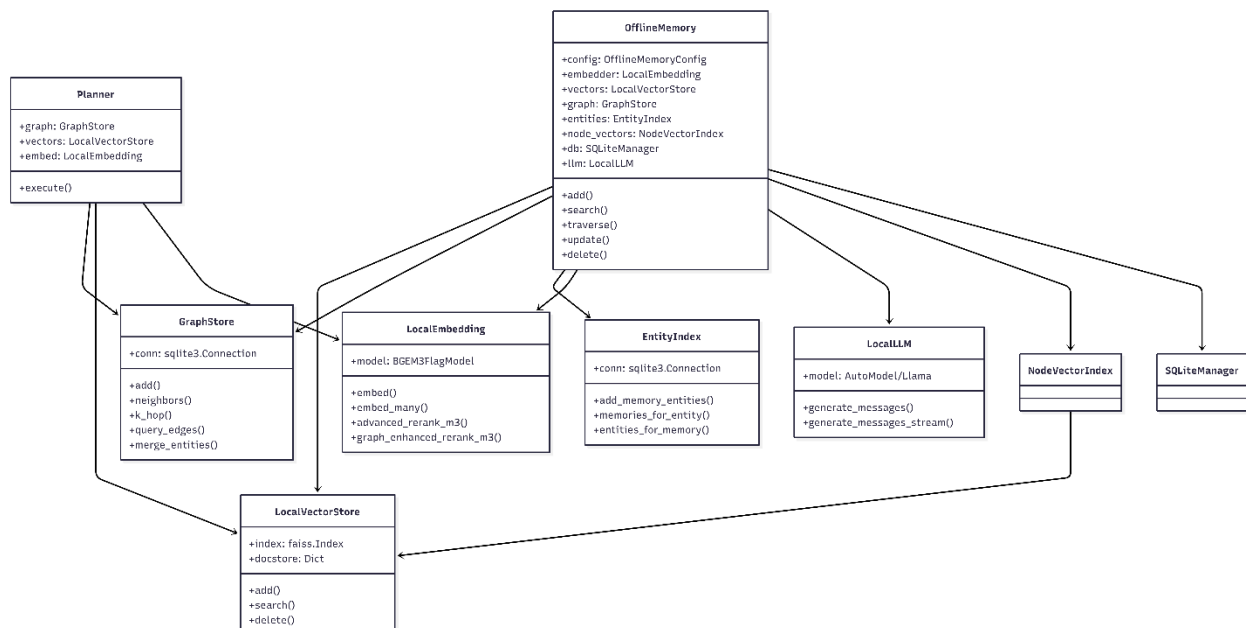


ARAISHA Memory System

Normal Chat Flow (Graph-Only)



Component Relationships



File Size Summary

<i>File</i>	<i>Lines</i>	<i>Purpose</i>
<i>memory.py</i>	<i>1604</i>	<i>Core orchestration</i>
<i>memory_system.py</i>	<i>1264</i>	<i>Alternative CLI</i>
<i>graph_store.py</i>	<i>762</i>	<i>Graph database</i>
<i>local_embedding.py</i>	<i>426</i>	<i>BGE-M3 embeddings</i>
<i>local_llm.py</i>	<i>380</i>	<i>LLM backend</i>
<i>visualize.py</i>	<i>274</i>	<i>PyVis export</i>
<i>vector_store.py</i>	<i>220</i>	<i>FAISS storage</i>
<i>storage.py</i>	<i>219</i>	<i>History DB</i>
<i>local_config.py</i>	<i>185</i>	<i>Configuration</i>
<i>dsl.py</i>	<i>144</i>	<i>Query parser</i>
<i>planner.py</i>	<i>103</i>	<i>DSL executor</i>
<i>algorithms.py</i>	<i>91</i>	<i>Graph algorithms</i>
<i>entity_index.py</i>	<i>91</i>	<i>Entity-memory index</i>
<i>node_vectors.py</i>	<i>48</i>	<i>Node embeddings</i>
<i>utils.py</i>	<i>25</i>	<i>Utilities</i>
<i>Total</i>	<i>~5,836</i>	

Key Design Decisions

1. **Unified Memory ID:** Same UUID used across FAISS, Graph, and EntityIndex
2. **Hybrid Scoring:** Weighted combination of vector similarity, graph overlap, recency, evidence
3. **BGE-M3 Multi-Modal:** Dense + Sparse + ColBERT for comprehensive matching
4. *A Traversal**: Possessive chain parsing for relational queries ("Alice's friend's father")
5. **Graph-Enhanced Reranking:** Relationship context injected into BGE-M3 scoring
6. **SQLite Everything:** Nodes, edges, entities, history all in SQLite for portability
7. **Offline-First:** All models run locally (GGUF, HuggingFace)